



AI-Assisted Development Blueprint: 24-Hour E-Commerce MVP

Objective: Build and deploy a custom-coded, functional e-commerce Minimum Viable Product (MVP) in a single day (24 hours) by heavily leveraging AI code generators, AI-assisted IDEs, and prompt-driven backend tools.

Target Stack: Next.js (React), Tailwind CSS, Supabase (PostgreSQL), and Stripe/Razorpay.

The AI Builder's Toolkit (Prerequisites)

To move at this speed, you will not write code from scratch. You will act as a **Director**, guiding AI tools to write the code for you.

1. **AI Code Editor: Cursor** (A fork of VS Code with deep AI integration) or **Windsurf**. You will use this to generate logic and fix bugs instantly.
2. **UI Generator: v0.dev** (by Vercel) or **Lovable / Bolt.new**. These generate production-ready React components from text prompts.
3. **Backend AI: Supabase**. It has an AI assistant that writes SQL schemas and Row Level Security (RLS) policies from plain English.
4. **Content & Assets: ChatGPT / Claude 3.5 Sonnet** (for copy) and **Midjourney / DALL-E 3** (for placeholder product images and logos).

Timeline & Execution Plan

Block 1: Foundation & Assets (Hours 1 - 3)

Goal: Have all text, images, and brand guidelines ready so you don't get stuck later.

1. **Asset Generation:** Prompt Midjourney for 10-15 high-quality, consistent product images.
2. **Copywriting:** Use Claude/ChatGPT to generate the website copy.

- *Prompt:* "I am building a D2C e-commerce brand selling [Product]. Generate a homepage hero headline, 3 value propositions, and 5 short product descriptions in a JSON format."
3. **Environment Setup:** Open Cursor, initialize a new Next.js project (`npx create-next-app@latest`), and configure Tailwind CSS.

Block 2: AI UI Generation (Hours 4 - 9)

Goal: Build the entire visual frontend without writing CSS.

1. **Generate Components with v0.dev or Bolt.new:**
 - *Prompt 1:* "A modern e-commerce navbar with a logo on the left, links in the middle, and a cart icon with a badge on the right. Minimalist design."
 - *Prompt 2:* "A product grid showing 4 items per row. Each card has an image, title, price, and an 'Add to Cart' button."
 - *Prompt 3:* "A sleek shopping cart slide-out drawer (sidebar) showing added items and a checkout button."
2. **Integrate into Cursor:** Copy the generated React code from v0 directly into your Next.js project in Cursor.
3. **Client State:** Ask Cursor's AI (using `Cmd+K` or `Ctrl+K`) to add a global cart state.
 - *Cursor Prompt:* "Implement Zustand for state management in this project. Create a store for the shopping cart that allows adding items, removing items, and calculating the total price. Connect it to the navbar and product cards."

Block 3: Backend & Database Generation (Hours 10 - 14)

Goal: Set up a live database and connect it to your frontend.

1. **Supabase AI Setup:** Go to Supabase, create a project, and open the SQL Editor AI.
2. **Prompt the Database:**
 - *Supabase AI Prompt:* "Create tables for an e-commerce store: `users` , `products` , and `orders` . Products should have title, price, description, and `image_url`. Orders should link to users and contain an

array of product IDs and a total status. Write policies to allow anyone to read products, but only authenticated users can insert orders."

3. **Seed Data:** Ask the AI to generate SQL insert statements using your generated product data.
4. **Connect Backend in Cursor:**
 - *Cursor Prompt:* "Install the `@supabase/supabase-js` client. Here are my API keys. Create a utility file to fetch all products from the `products` table and map them to my product grid component."

Block 4: Payments & Checkout Logic (Hours 15 - 19)

Goal: Allow users to actually pay for the items.

1. **Stripe/Razorpay Integration:** Do not build a custom checkout form. Use Stripe Checkout or Razorpay Payment Links for speed.
2. **Cursor API Route Generation:**
 - *Cursor Prompt:* "Create a Next.js App Router API endpoint at `/api/checkout`. It should receive an array of product IDs and quantities from the frontend cart. Use the Stripe Node.js SDK to create a Checkout Session and return the URL. Here is the Stripe secret key."
3. **Connect Frontend:** Ask Cursor to update your "Checkout" button in the cart drawer to call this new API route and redirect the user.

Block 5: Debugging & Polishing (Hours 20 - 23)

Goal: Fix the inevitable bugs caused by stitching code together.

1. **AI Error Resolution:** When you run the app (`npm run dev`) and hit an error, do not Google it. Copy the exact error message and paste it into Cursor Chat.
 - *Cursor Prompt:* "I am getting this error when clicking checkout: [paste error]. Look at my `checkout.ts` and `cartStore.ts` files and fix the bug."
2. **Mobile Responsiveness:** Ask Cursor to review your pages.

- *Cursor Prompt:* "Review `page.tsx`. Ensure all Tailwind classes are fully mobile-responsive. Make the grid 1 column on mobile and 4 columns on desktop."

Block 6: Deployment (Hour 24)

Goal: Push to the live internet.

1. **Vercel:** Push your code to a GitHub repository.
2. Connect the repo to Vercel (the creators of Next.js).
3. Add your Supabase and Stripe API keys to the Vercel Environment Variables.
4. Click Deploy.

Crucial Mindset for 1-Day AI Development

- **Don't write, Prompt:** If you are typing standard boilerplate code, you are losing time. Use `Cmd+K` in Cursor to generate logic.
- **Accept "Good Enough":** You will not have perfect animations or complex user profiles in 24 hours. The goal is to prove people will click "Buy".
- **Let AI Read the Docs:** If integrating a 3rd party tool (like a specific shipping API), use Cursor's "@Docs" feature to let the AI read the documentation and write the integration for you.